

# 字符串格式化输出

```
# 获取用户输入并转为整数
num = int(input("请输入一个十进制整数: "))

# 使用 % 格式化输出二进制、八进制、十六进制
print("使用%%格式化: ")
print("二进制: %s" % bin(num))
print("八进制: %s" % oct(num))
print("十六进制: %s" % hex(num))

# 使用 format() 方法
print("\n使用format()格式化: ")
print("二进制: {}".format(bin(num)))
print("八进制: {}".format(oct(num)))
print("十六进制: {}".format(hex(num)))

# 使用 f-string
print("\n使用f-string格式化: ")
print(f"二进制: {bin(num)}")
print(f"八进制: {oct(num)}")
print(f"十六进制: {hex(num)}")
```

E.G.1

```
name = "小明"
score = 95
print("学生 %s 的成绩是 %d 分。" % (name, score))
```

## replace() 方法

### ☒ 基本语法:

```
str.replace(old, new, count)
```

- old: 要被替换的子串
- new: 用于替换的新子串
- count (可选): 替换的次数, 默认全部替换

注意 python 中字符串是不可变的类型:

E.G.1

替换敏感词

```
sensitive_character = '你好' # 敏感词库
test_sentence = input('请输入一段话:\n')
## 请使用 ** 替换敏感词 "你好" ##
test_sentence = test_sentence.replace(sensitive_character, '**')
print(test_sentence)
```

## 列表推导式

```
[
    [0 for x in range(5)] for y in range(5)
]
```

在 2025.06.28 11:08:37 于 13ms内执行

```
[[0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0]]
```

```
[5 for x in range(5)]
```

在 2025.06.28 11:08:58 于 8ms内执行

```
[5, 5, 5, 5, 5]
```

```
[
     [1, 1, 2, 2] for x in range(2)
]
```

## chr() 跟 ord()

```
chr(97), ord('a'), chr(97) == 'a', ord('a') == 97
```

-> ('a', 97, True, True)

## 元组

元组是不可变序列, 通过 `()` 或 `tuple()` 创建

```
[3] 1 a = "12", 3, 4, [1, 2]
    2 b = ("12", 3, 4, [1, 2])
    3 a, b, a == b
    在 2025.06.28 11:49:14 于 8ms内执行
```

```
(( '12', 3, 4, [1, 2]), ( '12', 3, 4, [1, 2]), True)
```

```
[4] 1 a = (1, ) # 只有一个元素也加上, 不不然无法跟数字 1 区分
    2 print(a)
    在 2025.06.28 11:49:21 于 6ms内执行
```

```
(1,)
```

```
[5] 1 lis = [1, 2, 3]
    2 a = tuple(lis)
    3 print(a)
    在 2025.06.28 11:49:26 于 7ms内执行
```

```
(1, 2, 3)
```

```
[6] 1 str = '123123'
    2 a = tuple(str)
    3 print(a)
    在 2025.06.28 11:49:29 于 5ms内执行
```

```
('1', '2', '3', '1', '2', '3')
```

## 元组的好处

- 元组可以在字典中作为 `键` 使用, 而列表不行
- 元组的访问和处理速度更快
- 元素可以作为很多内置函数和方法的返回值

## 列表的复制

切片或者 `copy` 都行, 但是列表不要直接 = 复制。

# 列表的复制

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
b = a
a[0] = 5
print(a, b) # 可以发现 b = a 接受的是同一个地址，浅拷贝
```

在 2025.04.03 09:15:37 于 3ms内执行

[5, 2, 3, 4, 5, 6, 7, 8, 9] [5, 2, 3, 4, 5, 6, 7, 8, 9]

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
b = a.copy()
a[0] = 5
print(a, b) # 使用 copy
```

在 2025.04.03 09:16:05 于 14ms内执行

[5, 2, 3, 4, 5, 6, 7, 8, 9] [1, 2, 3, 4, 5, 6, 7, 8, 9]

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
b = a[::]
a[0] = 5
print(a, b) # 切片也可以，切片产生新对象
```

在 2025.04.03 09:16:13 于 10ms内执行

[5, 2, 3, 4, 5, 6, 7, 8, 9] [1, 2, 3, 4, 5, 6, 7, 8, 9]

## 删除元素

```
1 x = [1, 3, 2, 4, 2, 4]
2 x = [num for num in x if num != 2]
3 x
```

Executed at 2025.06.28 11:59:17 in 6ms

[1, 3, 4, 4]

# 误区

1. py 中 `print('' '')` 多行打印在部分评测集可能不稳定，导致输出错误；
2. py 中没有 `double()` 这个函数，这是 C++ 的，要用 `float()`。
3. `string.isdigit()` 判断字符串是否由纯数字组成。
4. `string.find(substr, l, r)` 在 `[l, r)` 查找子串。找不到返回 `-1`。
5. `lis.sort()` 原地排序；`sorted(lis)` 返回 `list`；`lis.pop(int index)` 删除元素，不加参数默认删最后一个。除了 `pop` 还有 `remove` 跟 `del`。

```
player_info = {}
li = []
print('输入quit表示选手成绩录入完毕')
while True:
    name = input("请输入选手名称：\n")
    if name == 'quit':
        break
    score = float(input("请输入选手票数：\n"))
    player_info[name] = score

li = sorted([
    (score, name) for name, score in player_info.items()
], reverse = True)

# 输出排名
for i in range(1, len(li) + 1):
    print(f"第{i}名：{li[i-1][1]}, 成绩为{li[i-1][0]}分")
```

## 6. Indentation Error

1. Python 的编译阶段：  
IndentationError 是在代码解析 (parse) 阶段 (即编译阶段) 抛出的错误，而不是在运行时 (runtime) 抛出的。
2. 当 Python 读取你的代码时，它会先检查语法和缩进是否正确。如果缩进错误 (比如 if 下面的 print 没有缩进)，Python 会直接报错并终止，而不会进入 try 块的执行阶段。
3. `try-except` 只能捕获运行时 (runtime) 异常，而 IndentationError 是语法错误 (SyntaxError 的子类)，在运行前就被检测到了。
4. IndentationError 是 SyntaxError 的子类：
5. 像 SyntaxError、IndentationError 这类错误，Python 会在代码执行前检查，因此无法用 try-except 捕获。

1. `randint(int l, int r)` 和 `random()` 一个返回 `[a,b]` 一个返回 `[0,1]`

6. `repr(obj)`：返回对象的官方 representation 形式。

7. `'a'.islower(), 'A'.isupper(), '12'.isnumeric(), ' '.isspace(), 'abc好'.isalpha()`

## 8. strip

需要注意的是，`chars` 参数是可选的。要是不传入这个参数，该方法就会默认去除字符串首尾的所有空白字符。

`strip` 就是从两端去掉所有  $c_i \in parameters : str$  的字符直到终止

```
1 s = '###1@@@1231'|
2 s.strip('#@1'), s.lstrip('#@'), s.rstrip('123@')
在 2025.06.28 11:36:28 于 5ms内执行

('23', '1@@@1231', '###')
```

#### 9. 忽略转义字符

加 `r` 忽略转义字符

```
[5] 1 a = r'\t \n \r \' \" \'a'
    2 print(a)
    3 a
在 2025.06.28 11:40:12 于 10ms内执行

\t \n \r \' \" \'a
'\\t \\n \\r \\\' \\\' \" \\'a'
```

10. `float` 最多输出 16 个数字, 当浮点数进行运算后, 最长可输出 17 个数字。然而, 计算机系统只能提供**15**个数字的准确性。最后一位计算机系统根据二进制计算结果确定, 有一定误差。